

Lab 1

The screenshot shows two PostgreSQL query windows. The left window displays the SQL for Transaction 1 (T1), which requests an X-lock, selects film_id and rental_rate for film_id = 1, and then updates rental_rate to 3. The right window displays the SQL for Transaction 2 (T2), which requests an X-lock, selects film_id and rental_rate for film_id = 1, and then updates rental_rate to 2. Both transactions are committed.

Transaction 1 (T1) SQL:

```
-- T1
BEGIN;
--Request X-lock
SELECT film_id, rental_rate
FROM film WHERE film_id = 1
FOR UPDATE;

--
UPDATE film SET rental_rate = 3
WHERE film_id = 1;
COMMIT;
```

Transaction 2 (T2) SQL:

```
-- T2
BEGIN;
-- Request a X-lock
SELECT film_id, rental_rate
FROM film WHERE film_id =1 FOR UPDATE;

--
UPDATE film SET rental_rate = 2
WHERE film_id =1;
COMMIT;
```

Data Output for T1:

film_id	rental_rate
1	2.00

T2 requests a lock after T1, so it must wait for T1 to finish committing before it can run.

The screenshot shows the same two PostgreSQL query windows. The left window shows the SQL for Transaction 1 (T1) and its successful completion. The right window shows the SQL for Transaction 2 (T2) and its successful completion. The data output for T2 shows that rental_rate has been updated to 3.00.

Transaction 1 (T1) SQL:

```
-- T1
BEGIN;
--Request X-lock
SELECT film_id, rental_rate
FROM film WHERE film_id = 1
FOR UPDATE;

--
UPDATE film SET rental_rate = 3
WHERE film_id = 1;
COMMIT;
```

Transaction 2 (T2) SQL:

```
-- T2
BEGIN;
-- Request a X-lock
SELECT film_id, rental_rate
FROM film WHERE film_id =1 FOR UPDATE;

--
UPDATE film SET rental_rate = 2
WHERE film_id =1;
COMMIT;
```

Data Output for T2:

film_id	rental_rate
1	3.00

T1 has finished committing, and T2 has been granted permission.

Lab 2

The screenshot shows a SQL IDE with two panes. The left pane displays Transaction T1, and the right pane displays Transaction T2. Below the panes, the 'Data Output' tab shows the initial state of a table.

Transaction T1 (Left Pane):

```
1 -- T1
2 BEGIN;
3 --Request X-lock
4 SELECT film_id, rental_rate
5 FROM film WHERE film_id = 1;
6
7
8
9
10 --
11 SELECT film_id, rental_rate
12 FROM film WHERE film_id = 1;
13 COMMIT;
```

Transaction T2 (Right Pane):

```
1 -- T2
2
3
4
5 BEGIN;
6 UPDATE film SET rental_rate = 5.99
7 WHERE film_id = 1;
8 COMMIT;
```

Data Output (Bottom):

Showing rows: 1 to 1 of 1

film_id	rental_rate
1	3.00

Read before the update, the rental rate value is 3.

The screenshot shows a SQL IDE with Transaction T1 in the query pane and the updated state of a table in the 'Data Output' tab.

Transaction T1 (Query Pane):

```
1 -- T1
2 BEGIN;
3 --Request X-lock
4 SELECT film_id, rental_rate
5 FROM film WHERE film_id = 1;
6
7
8
9
10 --
11 SELECT film_id, rental_rate
12 FROM film WHERE film_id = 1;
13 COMMIT;
```

Data Output (Bottom):

Showing rows: 1 to 1 of 1

film_id	rental_rate
1	5.99

Read again after the update, and the rental_rate receives the new value.

Lab 3

Query	Query History	Query	Query History
<pre> 1 -- T1 2 -- Step 1 3 BEGIN; 4 UPDATE film SET rental_rate = 2.99 5 WHERE film_id = 100; 6 7 -- Step 3 8 UPDATE film SET rental_rate = 3.99 9 WHERE film_id = 200; 10 -- Blocks (Waiting for B) </pre>		<pre> 1 -- T2 2 3 4 5 6 -- Step 2 7 BEGIN; 8 UPDATE film SET rental_rate = 2.99 9 WHERE film_id = 200; 10 11 -- Step 4 12 UPDATE film SET rental_rate = 3.99 13 WHERE film_id = 100; 14 -- BOOM! ERROR : deadlock detected </pre>	
Data Output	Messages	Data Output	Messages
UPDATE 1		UPDATE 1	
Query returned successfully in 97 msec.		Query returned successfully in 118 msec.	

T1 and T2 apply an X-lock on IDs 100 and 200 of the film table.

```
6 -- Step 3
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 200;
9 -- Blocks (Waiting for B)
10
6 -- Step 2
7 BEGIN;
8 UPDATE film SET rental_rate = 2.99
9 WHERE film_id = 200;
10
11 -- Step 4
12 UPDATE film SET rental_rate = 3.99
13 WHERE film_id = 100;
14 -- BOOM! ERROR : deadlock detected
```

Data Output	Messages	Notifications
Total rows:	Waiting for the query to complete... 00:00:19.524	CRLF Ln 7, Col 1

Data Output	Messages	Notifications
Total rows:	Waiting for the query to complete... 00:00:00.232	

T1 and T2 are deadlocked because both have locked each other's rows and neither has completed a commit.

```
UPDATE 1
Query returned successfully in 20 secs 360 msec.

ERROR: deadlock detected
Process 10912 waits for ShareLock on transaction 1216; blocked by process 54540.
Process 54540 waits for ShareLock on transaction 1217; blocked by process 10912.

SQL state: 40P01
Detail: Process 10912 waits for ShareLock on transaction 1216; blocked by process 54540.
Process 54540 waits for ShareLock on transaction 1217; blocked by process 10912.
Hint: See server log for query details.
Context: while locking tuple (5,12) in relation "film"
```

Postgres cancels T2's process to allow T1 to run.

Lab 4

```
-- T1
-- Step 1
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- Locks the row proactively

-- Step 3
INSERT INTO rental
(rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;
```

Output Messages Notifications

Inventory_Id	film_id	store_id	last_update
[PK] integer	smallint	smallint	timestamp without time zone
10	2	2	2006-02-15 10:09:17

rows: 1 to 1 Page No: 1 of 1

rows: 1 Query complete 00:00:00.065 CRLF Ln 3, Col 1 Total rows: Waiting for the query to complete... 00:00:02.325

T1 is holding an X-lock on ID 10, so T2 must wait.

```
-- Step 3
INSERT INTO rental
(rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;
```

Output Messages Notifications

returned successfully in 57 msec.

```
1 -- T2
2
3
4
5
6 -- Step 2
7 BEGIN;
8 SELECT * FROM inventory
9 WHERE inventory_id = 10 FOR UPDAT
10 -- BLOCKS IMMEDIATELY
11
12 -- Step 4
13 -- Unblocks after A commits.
14 -- App logic detects it's already
15 ROLLBACK;
```

Data Output Messages Notifications

ROLLBACK

Query returned successfully in 58 msec.

T1 has updated; T2 is unblocked, but the app logic checks and detects that the disc has already been rented, so it immediately rolls back all commands to prevent two people from renting the same disc at the same time.